ANKARA ÜNİVERSİTESİ BİLGİSAYAR MÜHENDİSLİĞİ
AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ
DERSİ 1. VE 2. PROJE RAPORU

Ad: Samet Soyad: YILDIRIMÇAKAR
Öğrenci Numarası: 21290144
Github: https://github.com/sametyildirim314

İÇİNDEKİLER

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü Raporu
1. Proje Özeti ve Amacı
2. Erişim Yönetimi ve Rol Bazlı Yetkilendirme (RBAC)
2.1. Temel Yetkilendirme Testi
2.2. Kurumsal Rollerin Tanımlanması
2.3. Tablo ve İşlem Bazlı Kısıtlamalar
2.4. Kullanıcı Atamaları
2.5. Test Süreci ve Hata Giderimi (Troubleshooting)
3. Bölüm 2: Veri Şifreleme
3.1. Kolon Bazlı Şifreleme (Column-Level Encryption) Uygulaması
3.2. Sertifika ve Simetrik Anahtar Yapılandırması
4. Bölüm 3: SQL Injection Testleri ve Korunma
4.1. Zafiyet Simülasyonu (Stored Procedure)
4.2. Parametreli Sorgular ile Güvenlik Katmanı Oluşturma
5. Bölüm 4: Audit Logları (Sistem İzleme)
5.1. Server Audit ve Log Yönetimi
5.2. Veritabanı İzleme Kurallarının Tanımlanması
6. Bölüm 5: Dinamik Veri Maskeleme (Dynamic Data Masking - DDM)
6.1. Veri Gizliliği ve UNMASK Yetkilendirmesi
7. Bölüm 6: Zaman Bazlı Erişim Kısıtlaması (Logon Trigger)
7.1. Mesai Saatleri Koruması ve Bağlantı Denetimi

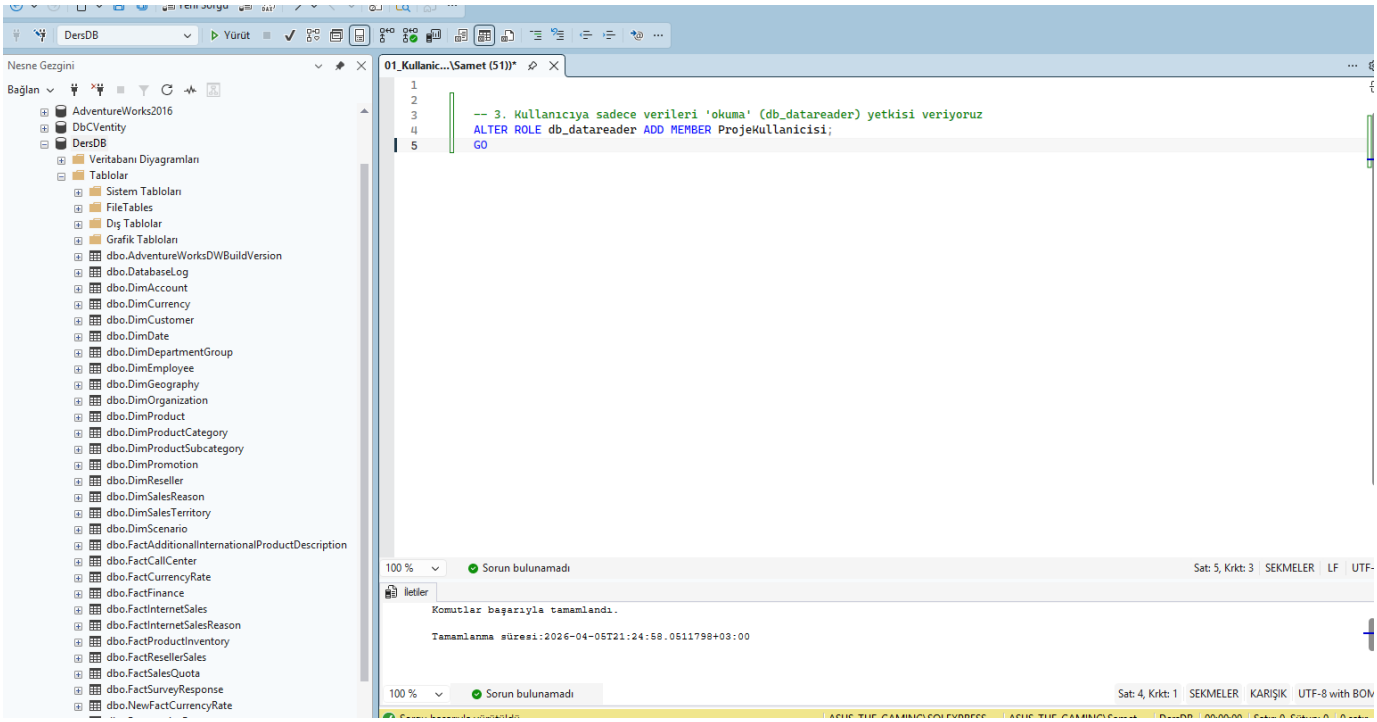
Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı
1. Proje Özeti ve Amacı

2. Bölüm 1: Kurtarma Modeli ve Temel Yedekleme Stratejisi
2.1. FULL Recovery Model Yapılandırması
2.2. Tam Yedek (Full Backup) ve Baseline Oluşturma
2.3. Fark Yedeği (Differential Backup) ile Veri Zinciri
3. Bölüm 2: Felaket Senaryosu Simülasyonu
3.1. Kritik Veri Girişi ve Zaman Damgası Kaydı
3.2. İnsan Hatası Kaynaklı Veri Kaybı (DELETE Senaryosu)
4. Bölüm 3: Zamana Dönüş ve Kurtarma Operasyonu (Point-in-Time Recovery)
4.1. Hata Yönetimi: Exclusive Access (İleti 3101) Çözümü
4.2. Tail-Log Backup ile Son Durumun Korunması
4.3. Restore Zincirinin İşletilmesi (Full -> Diff -> Tail-Log)
4.4. STOPAT Parametresi ile Hassas Kurtarma
5. Sonuç ve Veri Doğrulama

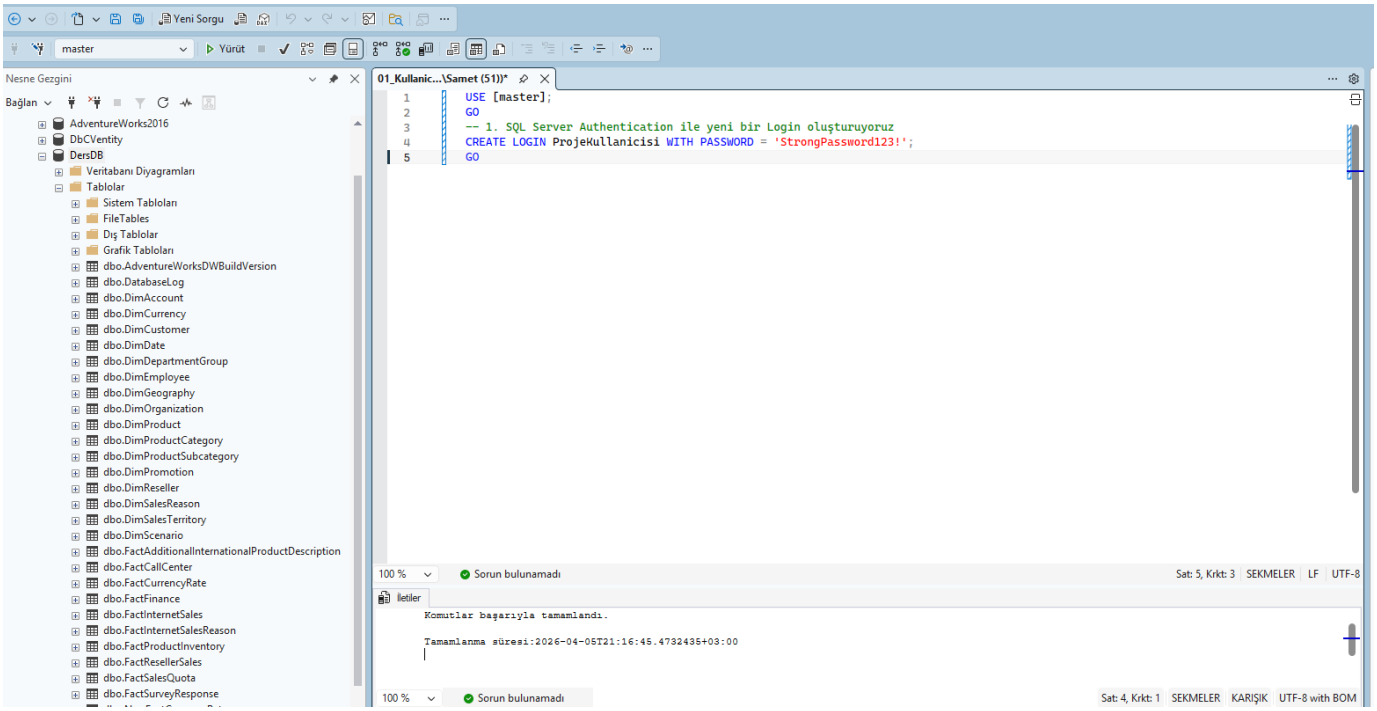
Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü Raporu

1. Proje Özeti ve Amacı Bu proje, veritabanı sistemlerinin güvenliğini sağlamak amacıyla kullanıcı erişimlerinin sınırlandırılması, hassas verilerin şifrelenmesi ve olası siber saldırılara karşı sistemin korunması adımlarını içermektedir. Proje kapsamında bir veritabanı yöneticisi (DBA) perspektifiyle dört temel güvenlik adımı MSSQL Management Studio (SSMS) üzerinden uygulanmıştır. Çalışmada hedef veritabanı olarak DersDB (AdventureWorks mimarisi temel alınarak) kullanılmıştır. Projenin her detayı raporlanmış ve versiyon kontrolü için Git kullanılarak belgelenmiştir.

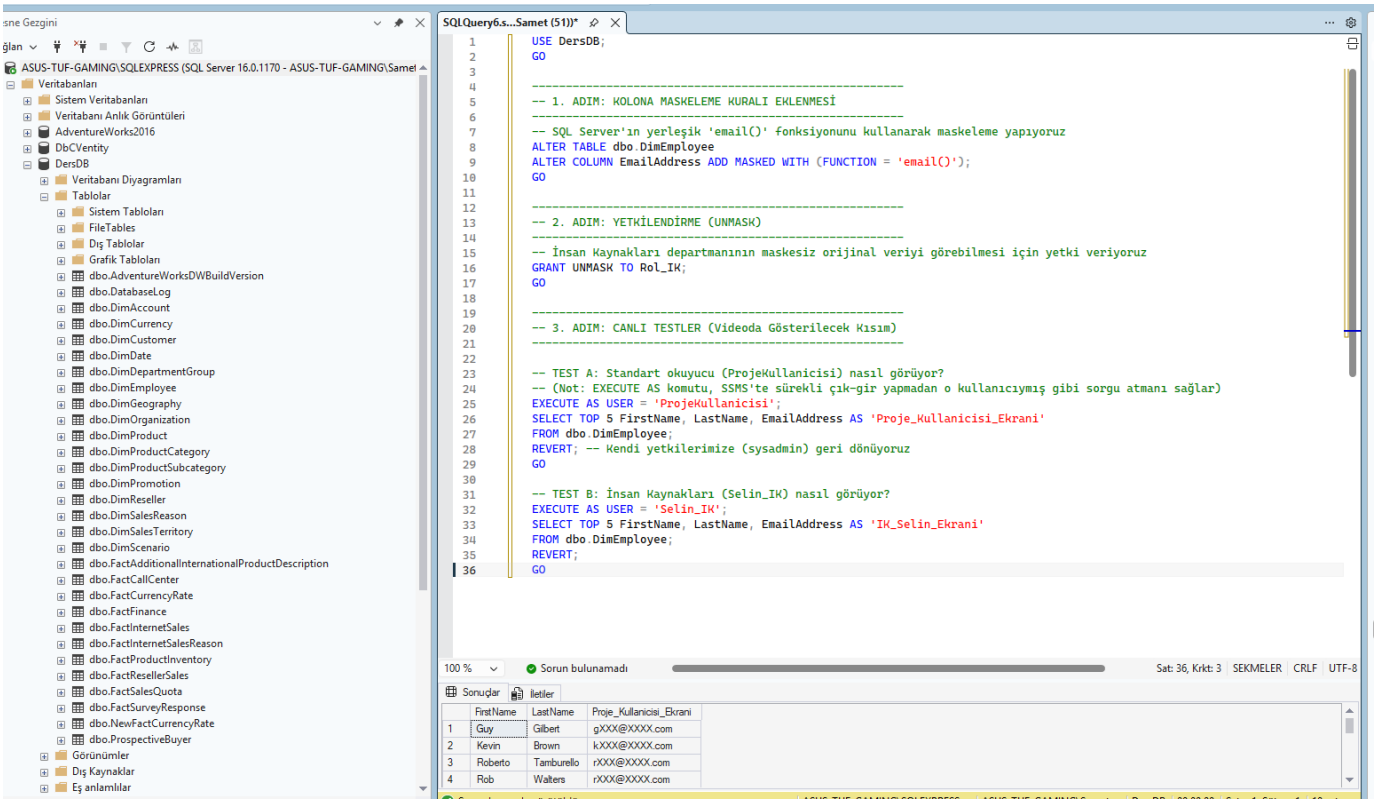
2. Erişim Yönetimi ve Rol Bazlı Yetkilendirme (RBAC) İlk aşamada, veritabanına yetkisiz erişimleri önlemek ve veritabanı güvenliği üzerine odaklanarak "en az yetki prensibini" (Principle of Least Privilege) kurumsal bir düzeyde uygulamak hedeflenmiştir. Kullanıcıların verilere erişim yetkilerini yönetmek amacıyla SQL Server Authentication yöntemi kullanılmış ve yapı, Rol Bazlı Erişim Kontrolü (Role-Based Access Control - RBAC) ile ölçeklendirilmiştir.



2.1. Temel Yetkilendirme Testi Sisteme ilk olarak temel bir güvenlik katmanı sağlamak amacıyla master veritabanı üzerinde ProjeKullanicisi adında bir giriş (Login) oluşturulmuştur. Bu kullanıcı DersDB veritabanında bir User olarak eşleştirilip sadece okuma yetkisi olan db_datareader rolüne atanarak tablo silme ve değiştirme yetkileri kısıtlanmıştır.



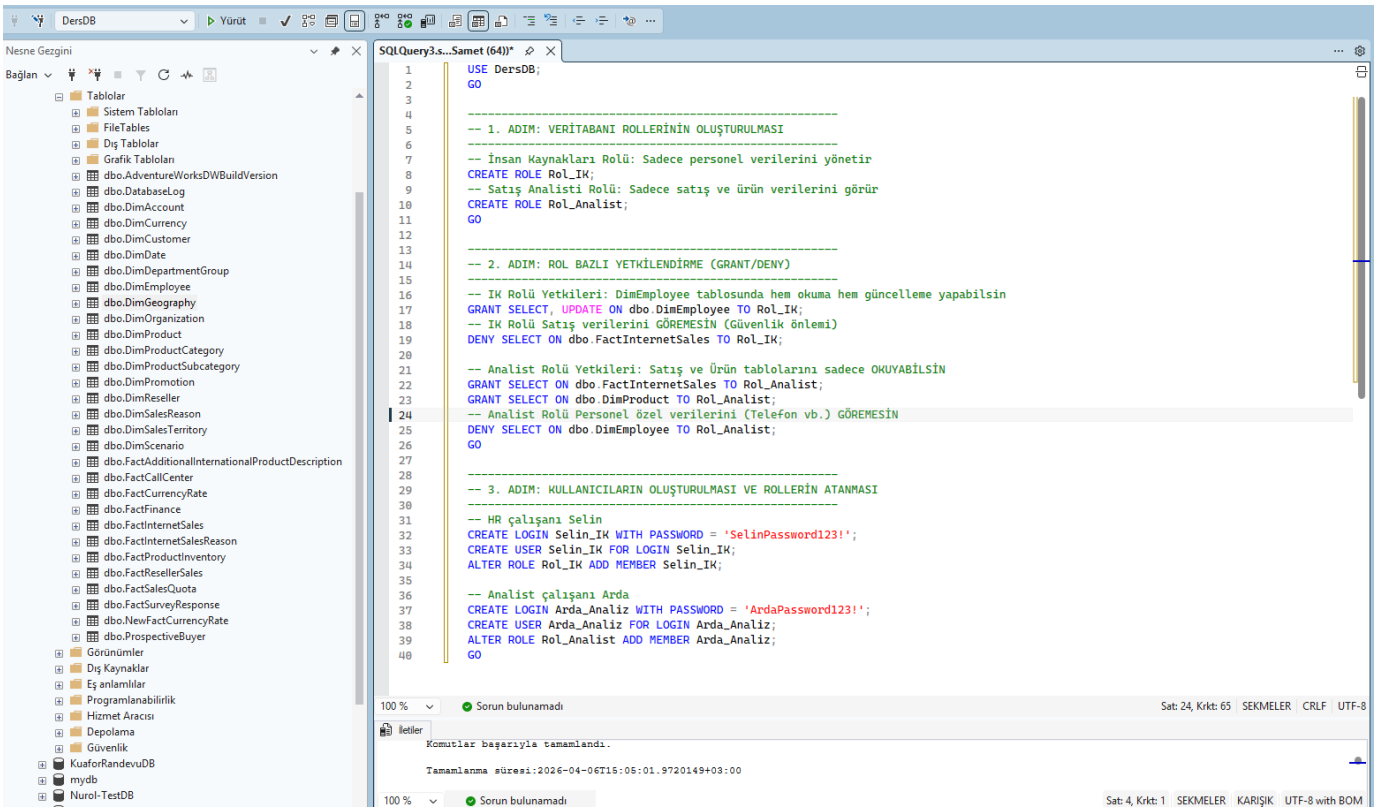
2.2. Kurumsal Rollerin Tanımlanması Sistemdeki veri izolasyonunu artırmak için departman bazlı roller tasarlanmıştır. "İnsan Kaynakları" (Rol_IK) ve "Satış Analisti" (Rol_Analist) adında iki farklı veritabanı rolü oluşturulmuştur.



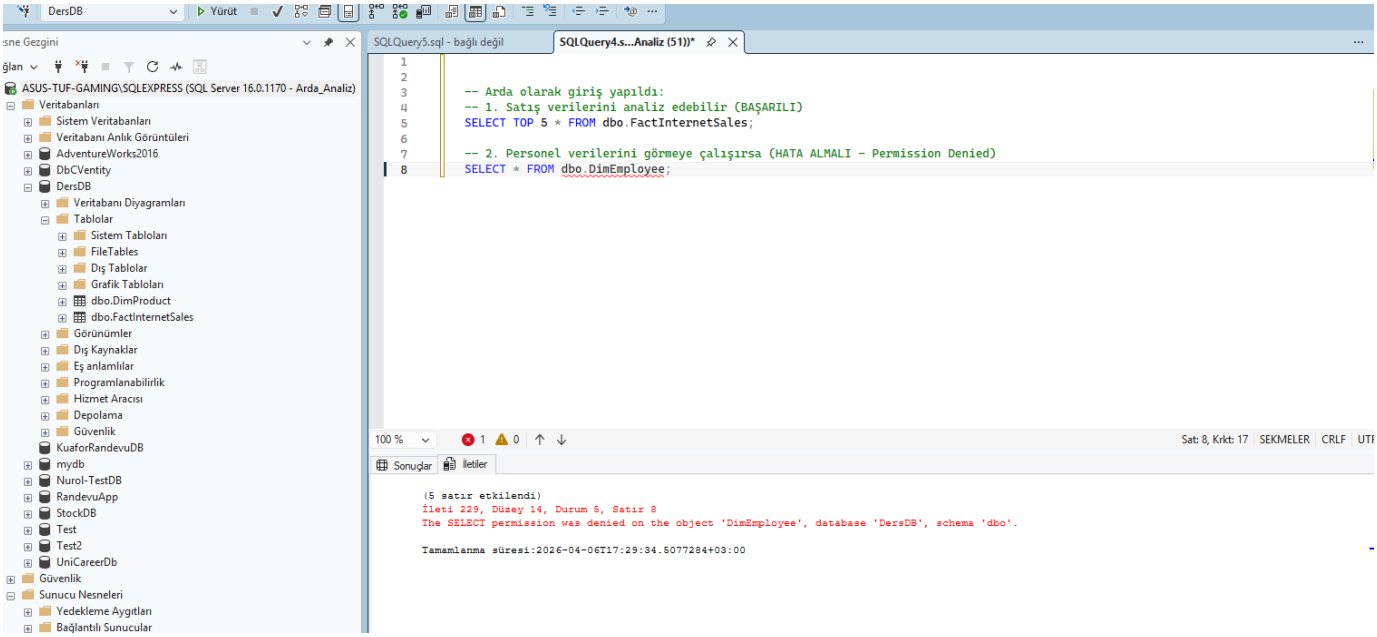
2.3. Tablo ve İşlem Bazlı Kısıtlamalar Rol_IK grubuna, sadece personel verilerini içeren DimEmployee tablosunda SELECT ve UPDATE yapma yetkisi verilmiş (GRANT), ancak satış verilerini görmemeleri için FactInternetSales tablosuna erişimleri açıkça engellenmiştir (DENY).

Rol_Analist grubuna ise, analiz yapabilmeleri için FactInternetSales ve DimProduct tablolarında sadece SELECT yetkisi verilmiş, personelin kişisel verilerini görmemeleri için DimEmployee tablosuna erişimleri reddedilmiştir (DENY).

2.4. Kullanıcı Atamaları Testleri gerçekleştirmek üzere Selin_IK ve Arda_Analiz adında yeni kullanıcılar oluşturularak ilgili rollere üye yapılmıştır.



2.5. Test Süreci ve Hata Giderimi (Troubleshooting) Test aşamasında Arda_Analiz kullanıcısı ile yeni bir sorgu penceresi açılarak FactInternetSales tablosuna SELECT işlemi denenmiştir. Bu denemede Invalid object name 'dbo.FactInternetSales' hatası ile karşılaşılmıştır. Hatanın nedeninin, SQL Server Management Studio'da (SSMS) yeni bir kullanıcıyla açılan sekmelerin varsayılan olarak sistemin ana veritabanı olan master dizininde başlaması olduğu tespit edilmiştir. master veritabanı içinde bu isimde bir tablo bulunmadığı için obje bulunamadı hatası alınmıştır.



Çözüm olarak sorgunun en başına USE DersDB; komutu eklenerek veritabanı bağlamı düzeltilmiştir. Çözüm sonrası sorgu başarıyla çalışmış ve Arda kullanıcısı satış verilerini listeleyebilmiştir. Aynı kullanıcı yetkisi olmayan DimEmployee tablosunu sorgulamaya çalıştığında ise sistem beklenen "The SELECT permission was denied on the object" hatasını fırlatarak kurulan güvenlik mimarisinin başarıyla çalıştığını kanıtlamıştır.

3. Bölüm 2: Veri Şifreleme Veritabanındaki hassas bilgilerin şifrelenmesi amacıyla sistem yapılandırılmıştır.

3.1. Kolon Bazlı Şifreleme (Column-Level Encryption) Uygulaması Kullanılan SQL Server Express sürümü fiziksel seviyede Transparent Data Encryption (TDE) desteklemediği için, mimari olarak daha spesifik ve güvenli olan Kolon Bazlı Şifreleme (Column-Level Encryption) yöntemi tercih edilmiştir.

3.2. Sertifika ve Simetrik Anahtar Yapılandırması Veritabanı seviyesinde EmployeeSertifikasi adında bir sertifika ve AES_256 algoritmasını kullanan bir simetrik anahtar (EmployeeAnahtari) oluşturulmuştur. dbo.DimEmployee tablosunda personellerin iletişim numaralarını barındıran Phone kolonu hedef alınmıştır. Bu tabloya SifreliTelefon adında VARBINARY tipinde yeni bir kolon eklenmiş ve mevcut telefon numaraları oluşturulan simetrik anahtarla şifrelenerek bu kolona aktarılmıştır. Bu sayede anahtara sahip olmayan hiçbir kullanıcı gerçek telefon numaralarına ulaşamaz hale getirilmiştir.

4. Bölüm 3: SQL Injection Testleri ve Korunma Veritabanının uygulamanın zafiyetlerinden kaynaklanan SQL injection saldırılarına karşı korunması için testler yapılmıştır.

4.1. Zafiyet Simülasyonu (Stored Procedure) Sisteme önce dışarıdan gelen metinleri doğrudan çalıştıran güvensiz bir Stored Procedure (sp_GuvenliksizArama) yazılmıştır. Bu prosedür üzerinden Smith' OR 1=1 -- mantıksal saldırısı yapılarak tüm tablonun yetkisiz bir şekilde listelendiği tespit edilmiştir.

4.2. Parametreli Sorgular ile Güvenlik Katmanı Oluşturma Zafiyeti gidermek için sorgular dinamik string birleştirme yerine Parametreli Sorgular (Parameterized Queries) kullanılarak (sp_GuvenliArama) yeniden yapılandırılmıştır. Aynı saldırı parametreli sorguya yapıldığında, veritabanı motoru metni bir SQL komutu olarak değil, sadece bir değer (string) olarak okuduğu için saldırı başarılı bir şekilde engellenmiştir.

5. Bölüm 4: Audit Logları (Sistem İzleme) Kullanıcı aktivitelerini izlemek ve şüpheli işlemleri tespit etmek için SQL Server Audit özellikleri kullanılmıştır.

5.1. Server Audit ve Log Yönetimi Sunucu düzeyinde bir "Server Audit" oluşturulmuş ve logların işletim sisteminde C:\Temp\ klasörüne yazılması sağlanmıştır.

5.2. Veritabanı İzleme Kurallarının Tanımlanması DersDB veritabanı düzeyinde bir "Database Audit Specification" oluşturularak, özellikle dbo.DimEmployee tablosu üzerindeki tüm okuma (SELECT) ve güncelleme (UPDATE) işlemleri izlemeye alınmıştır. Sisteme gönderilen test sorgularının başarıyla log dosyasına (event_time, action_id, statement) yazıldığı teyit edilmiştir.

6. Bölüm 5: Dinamik Veri Maskeleme (Dynamic Data Masking - DDM) Fiziksel şifrelemeye ek olarak, veri gizliliğini (Data Privacy) sağlamak ve hassas verilerin ekranlarda yetkisiz personellere görünmesini engellemek amacıyla Dinamik Veri Maskeleme (DDM) teknolojisi kullanılmıştır.

6.1. Veri Gizliliği ve UNMASK Yetkilendirmesi dbo.DimEmployee tablosunda bulunan EmailAddress kolonu, SQL Server'ın yerleşik email() fonksiyonu kullanılarak maskelenmiştir. Kurulan Rol Bazlı Erişim Kontrolü (RBAC) ile entegre çalışacak şekilde, İnsan Kaynakları rolüne (Rol_IK) maskeyi kaldırma (UNMASK) yetkisi tanımlanmıştır.

Test sürecinde standart okuma yetkisine sahip ProjeKullanicisi hesabı ile sorgu atıldığında e-posta adreslerinin sXXX@XXXX.com formatında maskelendiği görülmüştür. Aynı sorgu Selin_IK kullanıcısı ile atıldığında ise UNMASK yetkisi sayesinde verilerin orijinal ve okunabilir formatta listelendiği test edilerek başarılı bir veri gizliliği mimarisi oluşturulmuştur.

7. Bölüm 6: Zaman Bazlı Erişim Kısıtlaması (Logon Trigger) Veritabanı güvenliğini en üst düzeye çıkarmak ve çalınan kimlik bilgileriyle (credential theft) yapılabilecek mesai dışı sızıntıları engellemek amacıyla sunucu düzeyinde Logon Trigger mimarisi kullanılmıştır.

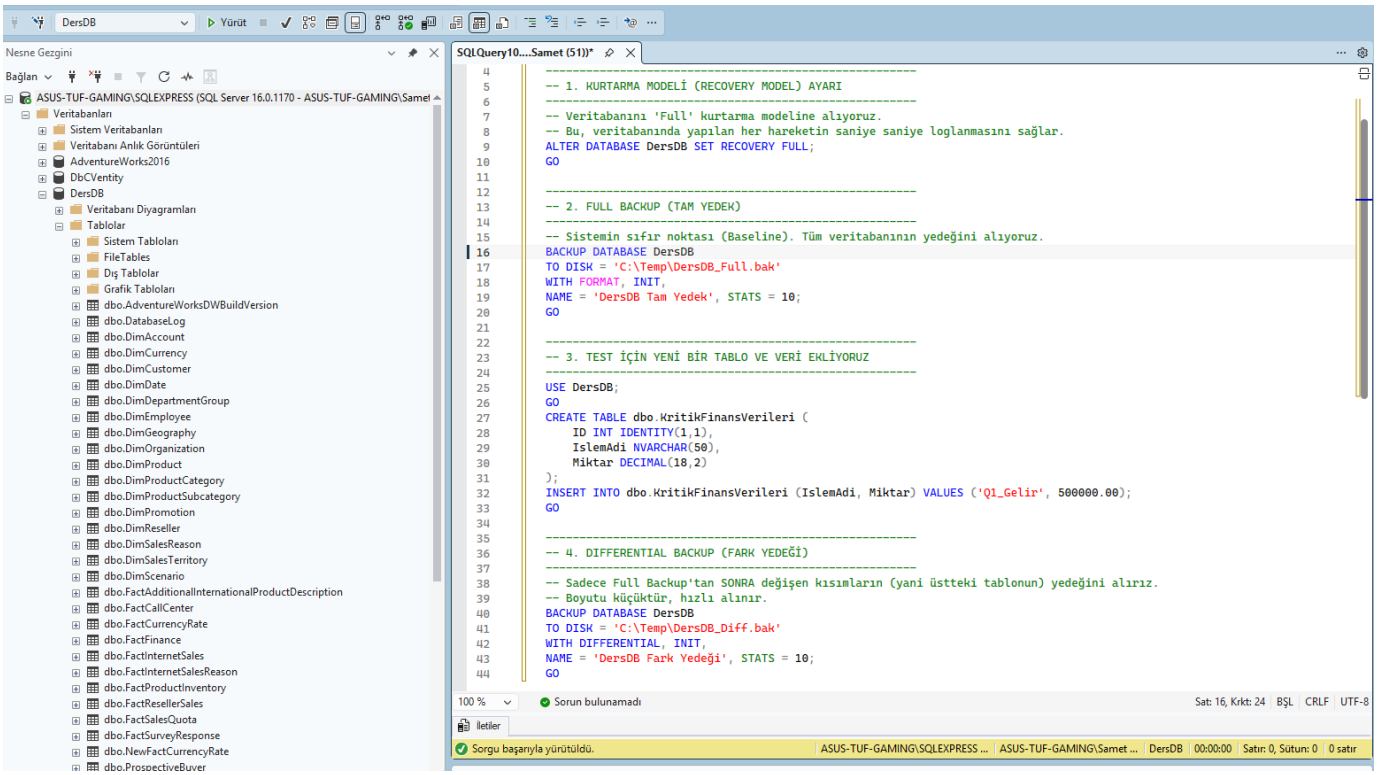
7.1. Mesai Saatleri Koruması ve Bağlantı Denetimi SQL Server üzerinde trg_MesaiSaatleriKorumasi adında, her giriş denemesinde tetiklenen bir güvenlik duvarı oluşturulmuştur. Sistem, ORIGINAL_LOGIN() fonksiyonu ile giriş yapan kullanıcıyı tespit edip GETDATE() ile sunucu saatini kontrol etmektedir.

Kural gereği 08:00 - 18:00 saatleri dışında giriş yapması yasaklanan Arda_Analiz hesabı ile mesai saatleri dışında sisteme bağlanılmaya çalışılmış ve doğru şifre girilmesine rağmen bağlantı isteğinin sunucu tarafından ROLLBACK komutuyla iptal edilip erişimin reddedildiği (Logon failed due to trigger execution) kanıtlanmıştır. Sistemin kilitlenmesini önlemek adına bu güvenlik duvarı DBA (sa) hesapları dışında tutulmuştur.

Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

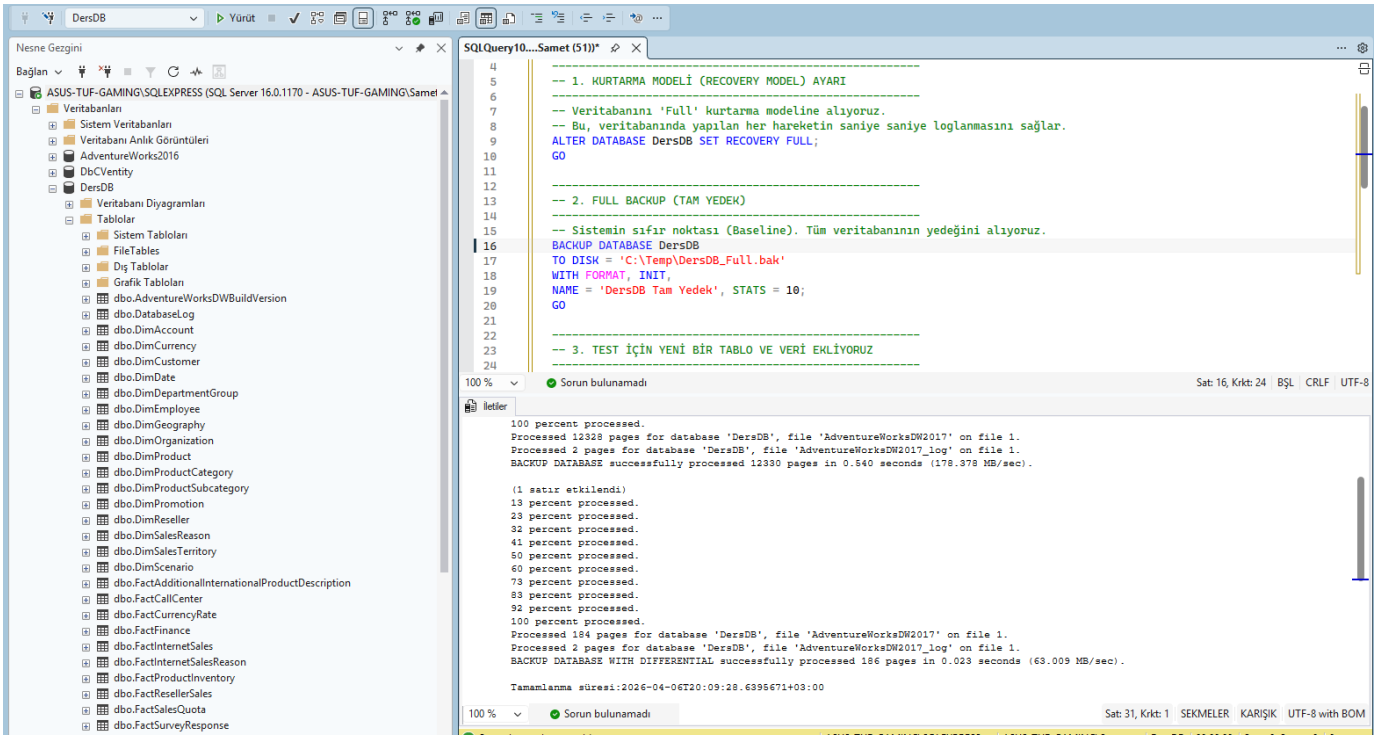
1. Proje Özeti ve Amacı Bu proje, kurumsal bir veritabanı sisteminde oluşabilecek veri kayıplarına karşı "Yedekleme ve Felaketten Kurtarma (Backup & Disaster Recovery)" stratejilerinin uygulanmasını amaçlamaktadır. Sadece standart yedekleme işlemleriyle kalınmamış; sisteme kritik verilerin girildiği, ardından insan hatasıyla (yanlış DELETE kullanımı) verilerin silindiği bir felaket senaryosu simüle edilmiştir. Sorunun çözümü için Transaction Log yedekleri kullanılarak "Zamana Dönüş (Point-in-Time Recovery)" işlemi gerçekleştirilmiş ve veri kaybı sıfıra indirilerek sistem başarıyla kurtarılmıştır. İşlemler DersDB veritabanı üzerinde gerçekleştirilmiştir.

2. Bölüm 1: Kurtarma Modeli ve Temel Yedekleme Stratejisi Veritabanında "Zamana Dönüş" yapılabilmesi için sistemin buna uygun konfigüre edilmesi ve temel yedekleme zincirinin (Backup Chain) oluşturulması sağlanmıştır.

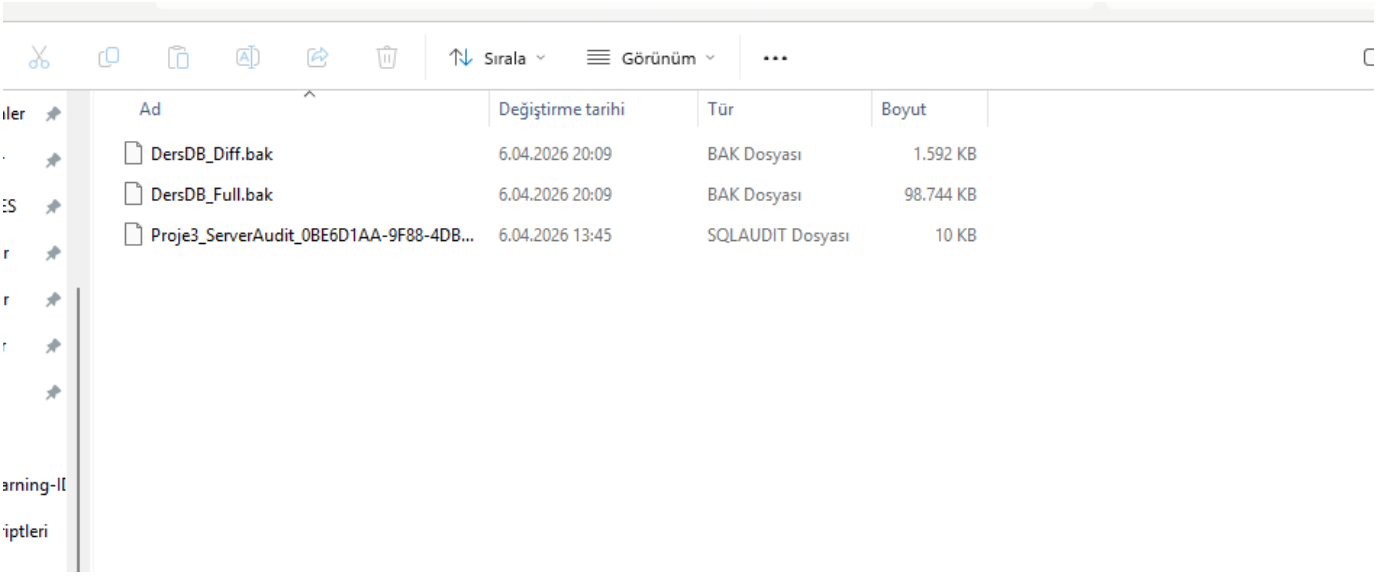


2.1. FULL Recovery Model Yapılandırması Veritabanında yapılan her bir işlemin saniye saniye günlüklere (log) yazılabilmesi için DersDB veritabanının kurtarma modeli FULL olarak ayarlanmıştır.

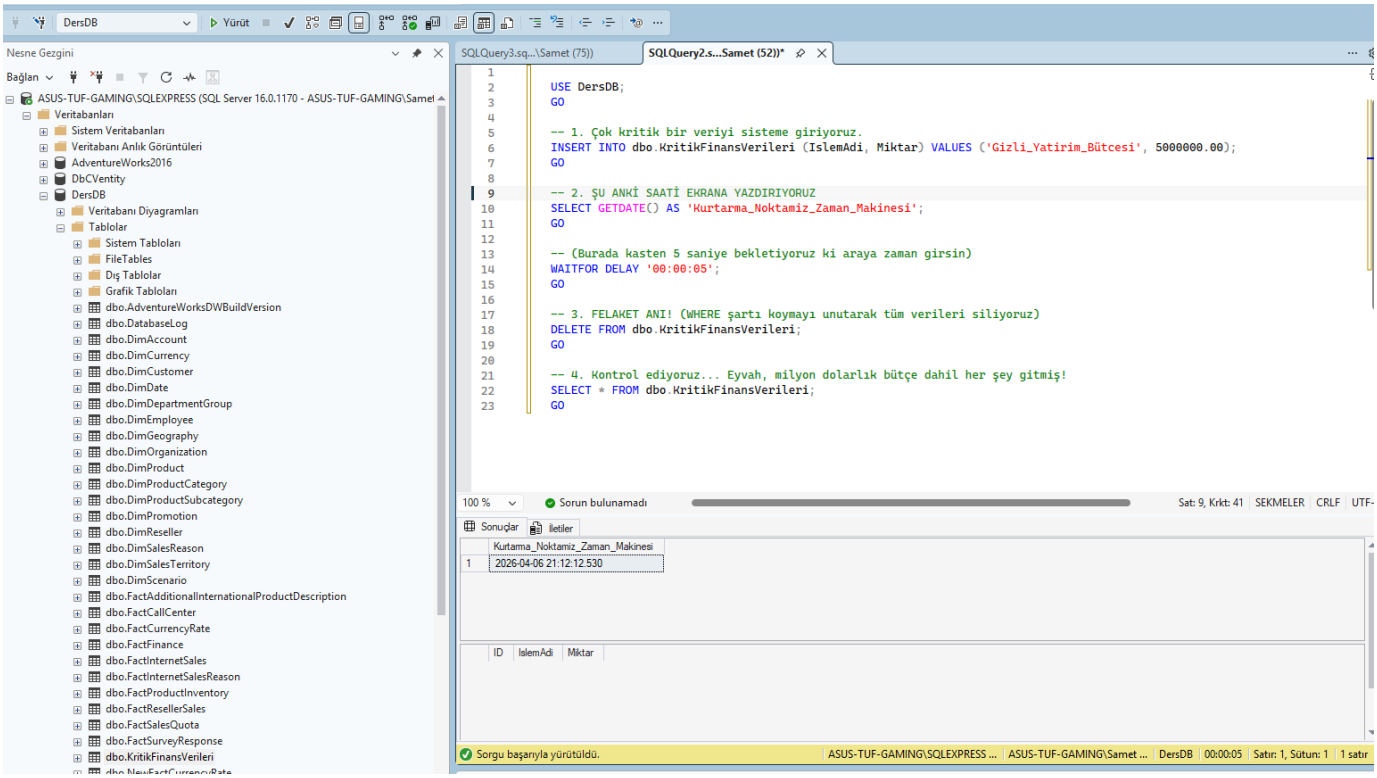
2.2. Tam Yedek (Full Backup) ve Baseline Oluşturma Sistemin sıfır noktasını (Baseline) oluşturmak amacıyla veritabanının DersDB_Full.bak adıyla tam yedeği alınmıştır.



2.3. Fark Yedeği (Differential Backup) ile Veri Zinciri Veritabanına KritikFinansVerileri adında yeni bir tablo eklenmiş ve içine ilk veriler girilmiştir. Tam yedekten sonra değişen bu yapıyı kaydetmek için DersDB_Diff.bak adıyla bir fark yedeği alınarak yedekleme zinciri desteklenmiştir.



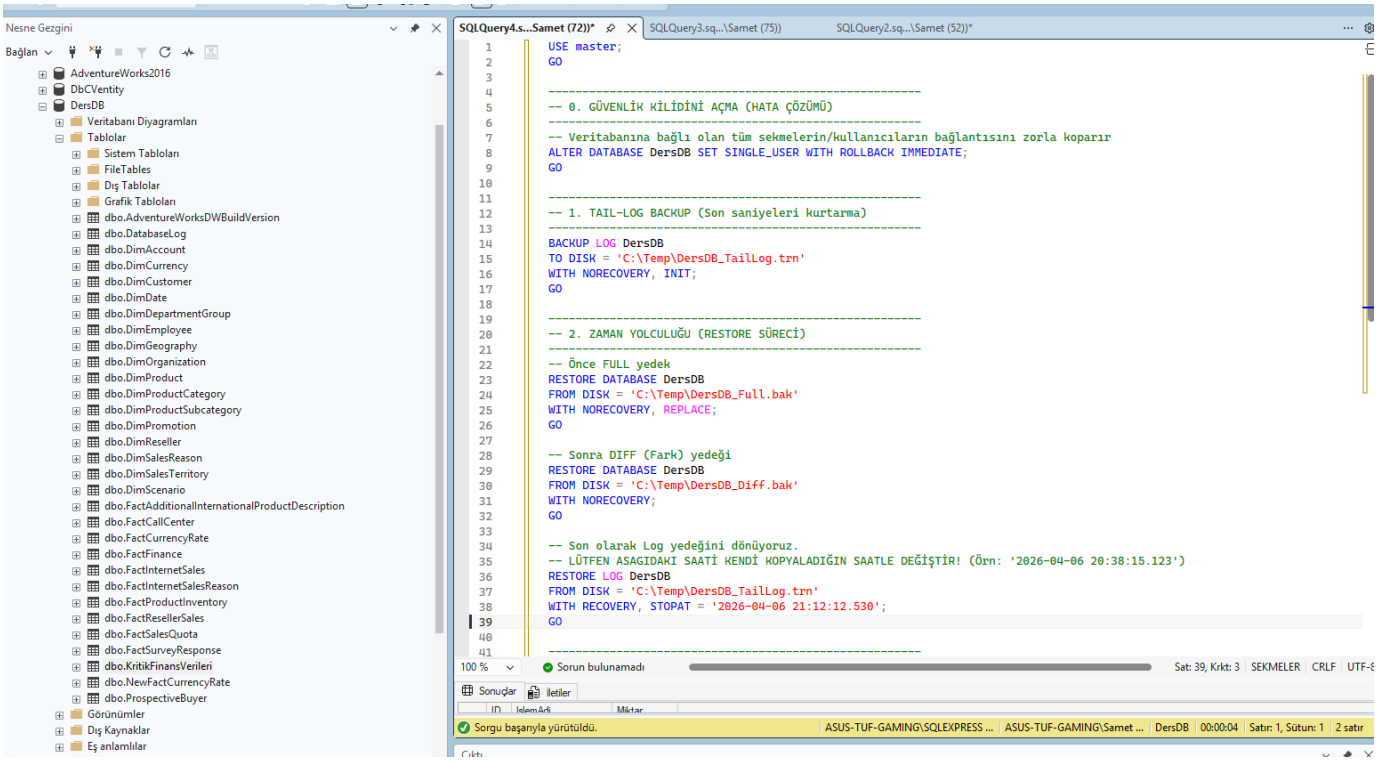
3. Bölüm 2: Felaket Senaryosu Simülasyonu Sistemin kurtarma yeteneklerini test etmek amacıyla bir felaket senaryosu (Disaster) kurgulanmıştır.



3.1. Kritik Veri Girişi ve Zaman Damgası Kaydı KritikFinansVerileri tablosuna milyon dolarlık yeni bir bütçe verisi eklenmiştir. İşlemin yapıldığı anın zaman damgası (GETDATE()), kurtarma hedefi olarak belirlenmek üzere sisteme yazdırılmış ve not alınmıştır.

3.2. İnsan Hatası Kaynaklı Veri Kaybı (DELETE Senaryosu) Sistemde 5 saniyelik bir gecikme (delay) yaratıldıktan sonra, WHERE koşulu unutulmuş bir DELETE komutu çalıştırılarak tablodaki tüm veriler kasten silinmiştir. Yapılan SELECT sorgusu ile tablonun tamamen boşaldığı ve kritik verinin kaybolduğu teyit edilmiştir.

4. Bölüm 3: Zamana Dönüş ve Kurtarma Operasyonu (Point-in-Time Recovery) Verilerin silinme anından tam 1 saniye öncesine dönerek sistemi kurtarmak için "Restore" operasyonu başlatılmıştır.



4.1. Hata Yönetimi: Exclusive Access (İleti 3101) Çözümü Kurtarma işlemi başlatılırken SSMS üzerinde açık kalan diğer bağlantılar nedeniyle İleti 3101: Exclusive access could not be obtained hatası alınmıştır. Veritabanı SINGLE_USER WITH ROLLBACK IMMEDIATE moduna alınarak kilitler ve tüm açık bağlantılar zorla kapatılmıştır.

4.2. Tail-Log Backup ile Son Durumun Korunması Veritabanının silinme anındaki son durumunu garantiye almak için NORECOVERY parametresi ile DersDB_TailLog.trn adında bir son işlem günlüğü yedeği alınmıştır.

4.3. Restore Zincirinin İşletilmesi (Full -> Diff -> Tail-Log) Restore operasyonu sırasıyla işletilmiştir. Öncelikle DersDB_Full.bak (Tam Yedek) NORECOVERY moduyla geri yüklenmiştir. Ardından üzerine DersDB_Diff.bak (Fark Yedeği) yine NORECOVERY moduyla yüklenmiştir.

4.4. STOPAT Parametresi ile Hassas Kurtarma Son olarak Tail-Log yedeği geri yüklenirken STOPAT parametresi kullanılarak, hedef zaman çizgisi silinme anının tam öncesine ayarlanmış ve veritabanı RECOVERY moduyla canlandırılmıştır.

5. Sonuç ve Veri Doğrulama Kurtarma işlemi tamamlandıktan sonra veritabanı tekrar MULTI_USER moduna alınmıştır. Yapılan SELECT sorgusu sonucunda, yanlışlıkla silinen milyon dolarlık bütçe verisinin eksiksiz bir şekilde geri geldiği kanıtlanmıştır.